

1. HTML5 Code to Display Audio and Video

```
<!DOCTYPE html>
<html>
<head><title>Media Example</title></head>
<body>
  <h2>Audio</h2>
  <audio controls>
    <source src="audio.mp3" type="audio/mpeg">
    Your browser does not support the audio element.
  </audio>

  <h2>Video</h2>
  <video width="320" height="240" controls>
    <source src="video.mp4" type="video/mp4">
    Your browser does not support the video tag.
  </video>
</body>
</html>
```

2. HTML5 Iframe Example

```
<!DOCTYPE html>
<html>
<head><title>Iframe Example</title></head>
<body>
  <h2>Google Page in Iframe</h2>
  <iframe src="https://www.google.com" width="600" height="400"></iframe>
</body>
</html>
```

3. Responsive Website Layout using Tailwind CSS

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100">
  <header class="bg-blue-600 text-white p-4 text-center">My Website</header>
  <div class="grid grid-cols-1 md:grid-cols-3 gap-4 p-4">
    <aside class="bg-white p-4">Sidebar</aside>
    <main class="bg-white p-4 col-span-2">Main Content</main>
  </div>
  <footer class="bg-blue-600 text-white text-center p-2">Footer</footer>
</body>
</html>
```

4. Responsive Pricing Table using Tailwind CSS

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://cdn.tailwindcss.com"></script>
```

```

</head>
<body>
  <div class="flex flex-col md:flex-row justify-center gap-6 p-6">
    <div class="border p-4 rounded-lg w-full md:w-1/3">
      <h2 class="text-xl font-bold mb-2">Basic</h2>
      <ul>
        <li>Feature A</li>
        <li>Feature B</li>
      </ul>
    </div>
    <div class="border p-4 rounded-lg bg-blue-100 w-full md:w-1/3">
      <h2 class="text-xl font-bold mb-2">Premium</h2>
      <ul>
        <li>Feature A</li>
        <li>Feature B</li>
        <li>Feature C</li>
      </ul>
    </div>
    <div class="border p-4 rounded-lg w-full md:w-1/3">
      <h2 class="text-xl font-bold mb-2">Enterprise</h2>
      <ul>
        <li>Feature A</li>
        <li>Feature B</li>
        <li>Feature C</li>
        <li>Feature D</li>
      </ul>
    </div>
  </div>
</body>
</html>

```

5. Registration Form with Validation using HTML5 and Tailwind CSS

```

<!DOCTYPE html>
<html>
<head>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="p-6">
  <form class="max-w-md mx-auto bg-white p-4 rounded shadow">
    <input class="border p-2 mb-2 w-full" type="text" placeholder="Name" required>
    <input class="border p-2 mb-2 w-full" type="email" placeholder="Email" required>
    <input class="border p-2 mb-2 w-full" type="password" placeholder="Password"
required minlength="6">
    <button class="bg-blue-500 text-white px-4 py-2" type="submit">Register</button>
  </form>
</body>
</html>

```

6. Responsive Product Card using Tailwind CSS

```

<!DOCTYPE html>
<html>
<head><script src="https://cdn.tailwindcss.com"></script></head>

```

```

<body>
  <div class="max-w-sm mx-auto p-4">
    <div class="border rounded-lg p-4 hover:shadow-lg transition">
      
      <h2 class="text-lg font-bold">Product Title</h2>
      <p>Description of the product</p>
      <p class="text-green-500 font-bold">$99</p>
      <button class="bg-blue-500 text-white px-4 py-2 mt-2">Add to Cart</button>
    </div>
  </div>
</body>
</html>

```

7. React Counter App using useState

```

import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

export default Counter;

```

8. React Form using useState and Console Logging

```

import React, { useState } from 'react';

function ContactForm() {
  const [form, setForm] = useState({ name: '', email: '', message: '' });

  const handleChange = e => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = e => {
    e.preventDefault();
    console.log(form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" onChange={handleChange} placeholder="Name" />
      <input name="email" onChange={handleChange} placeholder="Email" />
      <textarea name="message" onChange={handleChange} placeholder="Message" />
      <button type="submit">Submit</button>
    </form>
  );
}

```

```
export default ContactForm;
```

9. Multi-step Form in React with Progress

```
import React, { useState } from 'react';

function MultiStepForm() {
  const [step, setStep] = useState(1);
  const [data, setData] = useState({ name: '', email: '', street: '', city: '', zip: ''
});

  const next = () => setStep(step + 1);
  const prev = () => setStep(step - 1);
  const handleChange = e => setData({ ...data, [e.target.name]: e.target.value });

  return (
    <div>
      <progress value={step} max="3" />
      {step === 1 && (
        <div>
          <input name="name" onChange={handleChange} placeholder="Name" />
          <input name="email" onChange={handleChange} placeholder="Email" />
          <button onClick={next}>Next</button>
        </div>
      )}
      {step === 2 && (
        <div>
          <input name="street" onChange={handleChange} placeholder="Street" />
          <input name="city" onChange={handleChange} placeholder="City" />
          <input name="zip" onChange={handleChange} placeholder="ZIP" />
          <button onClick={prev}>Back</button>
          <button onClick={next}>Next</button>
        </div>
      )}
      {step === 3 && (
        <div>
          <pre>{JSON.stringify(data, null, 2)}</pre>
          <button onClick={prev}>Back</button>
          <button onClick={() => alert('Form submitted!')}>Submit</button>
        </div>
      )}
    </div>
  );
}

export default MultiStepForm;
```

11. RESTful API for E-commerce Products using Node.js + Express

```
// server.js
const express = require('express');
const app = express();
app.use(express.json());
```

```

let products = [];
let id = 1;

app.post('/products', (req, res) => {
  const product = { id: id++, ...req.body };
  products.push(product);
  res.status(201).json(product);
});

app.get('/products/:id', (req, res) => {
  const product = products.find(p => p.id == req.params.id);
  res.json(product || { message: 'Product not found' });
});

app.get('/products', (req, res) => {
  res.json(products);
});

app.put('/products/:id', (req, res) => {
  let index = products.findIndex(p => p.id == req.params.id);
  if (index !== -1) {
    products[index] = { ...products[index], ...req.body };
    res.json(products[index]);
  } else {
    res.status(404).json({ message: 'Product not found' });
  }
});

app.delete('/products/:id', (req, res) => {
  products = products.filter(p => p.id != req.params.id);
  res.json({ message: 'Product deleted' });
});

app.listen(3000, () => console.log('API running on port 3000'));

```

12. GraphQL Query in PHP using GraphQL-PHP Library

```

<?php
require 'vendor/autoload.php';

use GraphQL\Type\Definition\Type;
use GraphQL\Type\Schema;
use GraphQL\GraphQL;

// Define a simple query type
$queryType = new \GraphQL\Type\Definition\ObjectType([
  'name' => 'Query',
  'fields' => [
    'hello' => [
      'type' => Type::string(),
      'resolve' => fn() => 'Hello, GraphQL!'
    ]
  ]
]);

```

```

]);

$schema = new Schema(['query' => $queryType]);

$query = '{ hello }';
$result = GraphQL::executeQuery($schema, $query);
$output = $result->toArray();

header('Content-Type: application/json');
echo json_encode($output);

```

13. PHP Match Expression for User Roles (PHP 8+)

```

<?php
$role = 'editor';

$permissions = match ($role) {
    'admin' => 'Full access',
    'editor' => 'Edit access',
    'viewer' => 'Read-only access',
    default => 'No access'
};

echo "Role: $role, Permissions: $permissions";

```

14. MongoDB CRUD Operations with PHP 8

```

<?php
require 'vendor/autoload.php';

$client = new MongoDB\Client;
$collection = $client->mydb->products;

// Insert
$collection->insertOne(['name' => 'Phone', 'price' => 199]);

// Read
$product = $collection->findOne(['name' => 'Phone']);
print_r($product);

// Update
$collection->updateOne(['name' => 'Phone'], ['$set' => ['price' => 299]]);

// Delete
$collection->deleteOne(['name' => 'Phone']);

```

15. Laravel CRUD API with API Resources

```

// Migration
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->text('description');
    $table->string('status');

```

```

        $table->timestamps();
    });

// Model
class Task extends Model {
    protected $fillable = ['title', 'description', 'status'];
}

// Resource
public function toArray($request) {
    return [
        'id' => $this->id,
        'title' => $this->title,
        'description' => $this->description,
        'status' => $this->status,
    ];
}

// Controller
public function index() {
    return TaskResource::collection(Task::all());
}
public function store(Request $request) {
    $task = Task::create($request->all());
    return new TaskResource($task);
}
public function show(Task $task) {
    return new TaskResource($task);
}
public function update(Request $request, Task $task) {
    $task->update($request->all());
    return new TaskResource($task);
}
public function destroy(Task $task) {
    $task->delete();
    return response()->json(['message' => 'Deleted']);
}

// API Routes
Route::apiResource('tasks', TaskController::class);

```